

0.0.1 Question 1a

“How much is a house worth?” Who might be interested in an answer to this question? **Please list at least three different parties (people or organizations) and state whether each one has an interest in seeing the housing price to be high or low.**

People that would be interested in this answer could be the seller, and the buyer, investors, local governments, and banks. The buyer and seller would obviously like to see how much a home is worth. Investors might be curious in order to determine if buying a home would be a worthy investment, or to determine how the general market is going. Local governments would also be interested in order to determine how much they would be getting based off of property and sales taxes. Banks also are the ones who loan money to the buyers, so they would also be interested to see how much a home is worth.

0.0.2 Question 1b

Which of the following scenarios strike you as unfair and why? You can choose more than one. There is no single right answer, but you must explain your reasoning. Would you consider some of these scenarios more (or less) fair than others? Why?

- A. A homeowner whose home is assessed at a higher price than it would sell for.
- B. A homeowner whose home is assessed at a lower price than it would sell for.
- C. An assessment process that systematically overvalues inexpensive properties and undervalues expensive properties.
- D. An assessment process that systematically undervalues inexpensive properties and overvalues expensive properties.

I think that all of these scenarios are unfair, but D would probably seem the most unfair to me. This is because I think if an inexpensive property sells for less than it could've, it hurts the seller more than it does any other group. If they are selling an inexpensive property I would assume they might be struggling financially more than people who own expensive properties, so the lost value in their home hurts them the most.

0.0.3 Question 1d

What were the central problems with the earlier property tax system in Cook County as reported by the Chicago Tribune ? And what were the primary causes of these problems? (Note: in addition to reading the paragraph above you will need to **read the [Project_CaseStudy.pdf](#) explaining the context and history of this dataset before answering this question**).

The main issues that was present in the property tax system in Cook County was valuations of homes were way off what they should've been. Instead of updating their methods to a more fair model to determine their property taxes, the county decided to keep their same method which kept a large amount of inaccurate valuations. This was caused from a model which valued working-class neighborhoods to have their homes valued higher than what they should've been, which increased property taxes. In other areas where the homeowners were more wealthy their homes were valued lower than they should've been, which gave them a little bit of a tax break.

0.1 Question 2a: More EDA

In good news you have already done a lot of EDA with this dataset in Project 1.

Before fitting any model, we should check for any missing data and/or unusual outliers.

Since we're trying to predict `Sale Price`, we'll start with that field.

Examine the `Sale Price` column in the `training_val_data` DataFrame and answer the following questions:

- 2ai). Does the `Sale Price` data have any missing, N/A, negative or 0 values for the data? If so, propose a way to handle this.
- 2aii). Does the `Sale Price` data have any unusually large outlier values? If so, propose a cutoff to use for throwing out large outliers, and justify your reasoning).
- 2aiii). Does the `Sale Price` data have any unusually small outlier values? If so, propose a cutoff to use for throwing out small outliers, and justify your reasoning.

Below are three cells. The first is a Markdown cell for you to write up your responses to all 3 parts above. The second two are code cells that are available for you to write code to explore the outliers and/or visualize the `Sale Price` data.

0.1.1 Question 2a i, ii, iii answer cell:** *Put your answers in this cell...*

- i) The `Sale Price` data doesn't have any missing or 0 values, but they do have a lot of values that are 1. In order to create an accurate model I think that we should be removing these data points from the training data since we don't have accurate information on the actual sale price.
- ii) Yes, the top 5 sale prices seem to be very high compared to the rest of the data, as we can see that the 5th highest home price is around 50% than the 6th highest home price. So I would remove the top 5 highest home prices from the data set.
- iii) Yes, there seems to be a lot of data points that have the sale price to be in the single thousand dollars, which doesn't make sense in determining the sale price of a home. So I would suggest a cutoff of less than \$10,000. # your code exploring Sale Price above this line

```
In [9]: train_copy = training_val_data.copy()
        train_copy = train_copy.sort_values(by = ["Sale Price"], ascending=False)
        train_copy.reset_index()
        train_copy["Sale Price"][:10]
        # optional extra cell for exploring code
```

```
Out[9]: 106057    71000000
        71953     28000000
```

37835	12700000
929	12250000
19702	12000000
157521	8490078
194905	8000000
8043	7800000
117989	7400000
193635	7144704

Name: Sale Price, dtype: int64

0.1.2 Question 5a: Choose an additional feature

It's your turn to choose another feature to add to the model. Choose one new **quantitative** (not qualitative) feature and create Model 3 incorporating this feature (along with the features we've already chosen in Model 2). Try to choose a feature that will have a large impact on reducing the RMSE and/or will improve your residual plots. This can be a raw feature available in the dataset, or a transformation of one of the features in the dataset, or a new feature that you create from the dataset (see Project 1 for ideas). In the cell below, explain what additional feature you have chosen and why. Justify your reasoning. There are optional code cells provided below for you to use when exploring the dataset to determine which feature to add.

Note: There is not one single right answer as to which feature to add, however you should make sure the feature decreases the Cross Validation RMSE compared to Model 2 (i.e. we want to improve the model, not make it worse!)

This problem will be graded based on your reasoning and explanation of the feature you choose, and then on your implementation of incorporating the feature.

NOTE Please don't add additional coding cells below or the Autograder will have issues. You do not need to use all the coding cells provided.

0.1.3 Question 5a Answer Cell:

In this cell, explain what feature you chose to add and why. Then give the equation for your new model (use Model 2 from above and then add an additional term).

```
In [40]: def process_data_candidates2(data):  
  
    # Remove Non-Market Sales  
    data = data[data["Pure Market Filter"]==1]  
  
    data["Log Sale Price"] = np.log(data["Sale Price"])  
  
    # Create Log Building Square Feet column  
    data["Log Building Square Feet"] = np.log(data["Building Square Feet"])  
  
    # Create Bedrooms  
    data = add_total_bedrooms(data)  
  
    # Update Roof Material feature with names  
    data = substitute_roof_material(data)  
  
    # Select columns for comparing residuals  
    data = data[['Log Building Square Feet', 'Roof Material', 'Bedrooms', 'Log Sale Price', 'V  
  
    return data
```

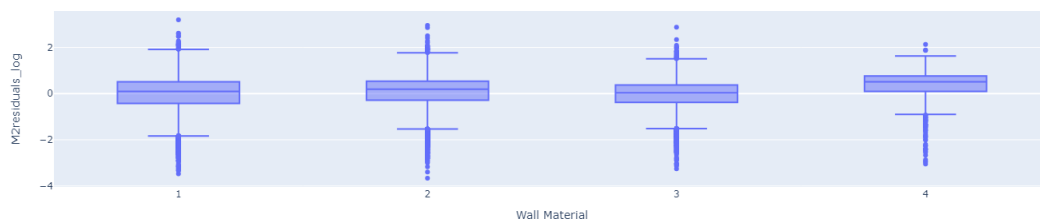
#Since our residuals are using the validation data, we will just examine these new features on

```
valid_comp2 = process_data_candidates2(valid)
```

```
valid_comp2 = valid_comp2.assign(M2residuals_log=Y_valid_m2 - Y_predict_valid_m2)
```

```
px.box(valid_comp2, x='Wall Material', y='M2residuals_log')
```

```
#px.box(valid_comp2, x='Construction Quality', y='M2residuals_log')  
# Show work in this cell exploring data to determine which feature to add
```



```
In [41]: #I am going to choose to add wall material as another feature. Both wall materials '2' and '4'  
#This means it might be a good model to fit for our data
```

```
# Optional code cell for additional work exploring data/ explaining which feature you chose.
```

```
In [42]: ...
```

```
# Optional code cell for additional work exploring data/ explaining which feature you chose.
```

```
Out[42]: Ellipsis
```

```
In [43]: ...
```

```
# Optional code cell for additional work exploring data/ explaining which feature you chose.
```

```
Out[43]: Ellipsis
```

```

In [44]: # Modeling Step 1: Process the Data

# Hint: You can either use your implementation of the One Hot Encoding Function
# from Project Part 1, or use the staff's implementation

from feature_func import *

# Optional: Define any helper functions you need for one-hot encoding above this line

def process_data_m3(data):

    # You should start by only keeping values with Pure Market Filter = 1

    data_copy = data[(data["Pure Market Filter"] == 1)]

    oh_enc = OneHotEncoder()
    oh_enc.fit(data_copy[["Roof Material"]]);
    arr = pd.DataFrame(oh_enc.transform(data_copy[["Roof Material"]]).todense(), columns = oh_
    data_copy = data_copy.join(arr)

    data_copy["Log Sale Price"] = np.log(data_copy["Sale Price"])
    data_copy["Log Buidling Square Feet"] = np.log(data_copy["Building Square Feet"])

    oh_enc.fit(data_copy[["Wall Material"]]);
    arr2 = pd.DataFrame(oh_enc.transform(data_copy[["Wall Material"]]).todense(), columns = oh_
    data_copy = data_copy.join(arr2)

    data_copy = data_copy[['Log Sale Price', 'Log Buidling Square Feet', 'Roof Material_1.0',

    return data_copy

# Use the same original train and valid datasets from 3a
# (otherwise the validation errors aren't comparable),
# Don't resplit the data.

# Process the data for Model 3
processed_train_m3 = process_data_m3(train)

processed_val_m3 = process_data_m3(valid)

# Create X (Dataframe) and Y (series) to use to train the model
X_train_m3 = processed_train_m3.drop(columns= "Log Sale Price")
Y_train_m3 = processed_train_m3["Log Sale Price"]

X_valid_m3 = processed_val_m3.drop(columns= "Log Sale Price")
Y_valid_m3 = processed_val_m3["Log Sale Price"]

```

```

# Take a look at the result
display(X_train_m3.head())
display(Y_train_m3.head())

display(X_valid_m3.head())
display(Y_valid_m3.head())

```

	Log Buidling Square Feet	Roof Material_1.0	Roof Material_2.0	\
130829	7.870166	1.0	0.0	
193890	7.002156	1.0	0.0	
30507	6.851185	1.0	0.0	
91308	7.228388	1.0	0.0	
131132	7.990915	1.0	0.0	

	Roof Material_3.0	Roof Material_4.0	Roof Material_5.0	\
130829	0.0	0.0	0.0	
193890	0.0	0.0	0.0	
30507	0.0	0.0	0.0	
91308	0.0	0.0	0.0	
131132	0.0	0.0	0.0	

	Roof Material_6.0	Wall Material_1.0	Wall Material_2.0	\
130829	0.0	1.0	0.0	
193890	0.0	1.0	0.0	
30507	0.0	0.0	1.0	
91308	0.0	1.0	0.0	
131132	0.0	0.0	0.0	

	Wall Material_3.0	Wall Material_4.0
130829	0.0	0.0
193890	0.0	0.0
30507	0.0	0.0
91308	0.0	0.0
131132	1.0	0.0

130829	12.994530
193890	11.848683
30507	11.813030
91308	13.060488
131132	12.516861

Name: Log Sale Price, dtype: float64

	Log Buidling Square Feet	Roof Material_1.0	Roof Material_2.0	\
50636	7.310550	1.0	0.0	
82485	7.325808	1.0	0.0	
193966	7.090077	1.0	0.0	
160612	7.281386	1.0	0.0	
7028	7.118016	1.0	0.0	

	Roof Material_3.0	Roof Material_4.0	Roof Material_5.0	\
--	-------------------	-------------------	-------------------	---

50636	0.0	0.0	0.0
82485	0.0	0.0	0.0
193966	0.0	0.0	0.0
160612	0.0	0.0	0.0
7028	0.0	0.0	0.0

	Roof Material_6.0	Wall Material_1.0	Wall Material_2.0 \
50636	0.0	1.0	0.0
82485	0.0	1.0	0.0
193966	0.0	0.0	1.0
160612	0.0	1.0	0.0
7028	0.0	0.0	1.0

	Wall Material_3.0	Wall Material_4.0
50636	0.0	0.0
82485	0.0	0.0
193966	0.0	0.0
160612	0.0	0.0
7028	0.0	0.0

50636	11.682668
82485	12.820655
193966	9.825526
160612	12.468437
7028	12.254863

Name: Log Sale Price, dtype: float64

In [45]: *# Modeling STEP 2: Create a Multiple Linear Regression Model*

Be sure to set fit_intercept to False, since we are incorporating one-hot-encoded data

linear_model_m3 = lm.LinearRegression(fit_intercept=False)

linear_model_m3.fit(X_train_m3, Y_train_m3)

your code above this line to create regression model for Model 3

Y_predict_train_m3 = linear_model_m3.predict(X_train_m3)

Y_predict_valid_m3 = linear_model_m3.predict(X_valid_m3)

In [46]: *# MODELING STEP 3: Evaluate the RMSE for your model*

Training and test errors for the model (in its units of Log Sale Price)

training_error_log[2] = rmse(Y_predict_train_m3, Y_train_m3.array)

validation_error_log[2]= rmse(Y_predict_valid_m3, Y_valid_m3.array)

```

Y_predict_train_m3_transformed = np.exp(Y_predict_train_m3)
Y_train_m3_transformed = np.exp(Y_train_m3)
Y_predict_valid_m3_transformed = np.exp(Y_predict_valid_m3)
Y_valid_m3_transformed = np.exp(Y_valid_m3)

# Training and test errors for the model (in its original values before the log transform)
training_error[2] = rmse(Y_predict_train_m3_transformed, Y_train_m3_transformed.array)
validation_error[2] = rmse(Y_predict_valid_m3_transformed, Y_valid_m3_transformed.array)

(print("3rd Model\nTraining RMSE (log): {}\nValidation RMSE (log): {}\n"
      .format(training_error_log[2], validation_error_log[2])))
)

(print("3rd Model \nTraining RMSE: {}\nValidation RMSE: {}\n"
      .format(training_error[2], validation_error[2])))
)

```

```

3rd Model
Training RMSE (log): 0.744904933298294
Validation RMSE (log): 0.7450430257989178

```

```

3rd Model
Training RMSE: 239921.59532982385
Validation RMSE: 245251.48623382236

```

In [47]: *# MODELING STEP 4: Conduct 5-fold cross validation for model and output RMSE*

```

linear_model_m3_cv = lm.LinearRegression(fit_intercept=False)

# Process the entire training_val dataset using the pipeline
processed_full_m3 = process_data_m3(training_val_data)

# Split into X and Y:
X_full_m3 = processed_full_m3.drop(columns = "Log Sale Price")
Y_full_m3 = processed_full_m3["Log Sale Price"]

# Run Cross Validation and Output RMSE (in units of Sale Price):
cv_error[2] = cross_validate_rmse(linear_model_m3_cv, X_full_m3, Y_full_m3)

print("3rd Model Cross Validation RMSE: {}".format(cv_error[2]))

```

```

3rd Model Cross Validation RMSE: 240855.4969580834

```

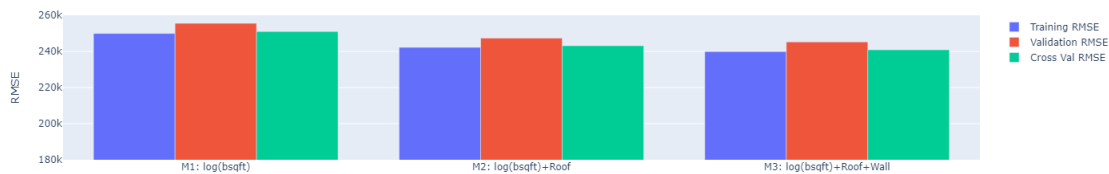
```
In [48]: # MODELING STEP 5: Add a name for your 3rd model describing the features
#and run this cell to Plot bar graph all 3 models
```

```
model_names[2] = "M3: log(bsqft)+Roof+Wall"
```

```
fig = go.Figure([
go.Bar(x = model_names, y = training_error, name="Training RMSE"),
go.Bar(x = model_names, y = validation_error, name="Validation RMSE"),
go.Bar(x = model_names, y = cv_error, name="Cross Val RMSE")
])
```

```
fig.update_yaxes(range=[180000,260000], title="RMSE")
```

```
fig
```



```
In [49]: # MODELING STEP 5 cont'd: Plot 2 side-by-side residual plots
#(similar to Question 3, for validation data)
```

```
fig, ax = plt.subplots(1,2, figsize=(15, 5))
```

```
x_plt1 = Y_predict_valid_m3
y_plt1 = Y_valid_m3.array - Y_predict_valid_m3
```

```
x_plt2 = Y_valid_m3.array
y_plt2 = Y_valid_m3.array - Y_predict_valid_m3
```

```
ax[0].scatter(x_plt1, y_plt1, alpha=.25)
ax[0].axhline(0, c='black', linewidth=1)
ax[0].set_xlabel(r'Predicted Log(Sale Price)')
ax[0].set_ylabel(r'Residuals: Log(Sale Price) - Predicted Log(Sale Price)');
```

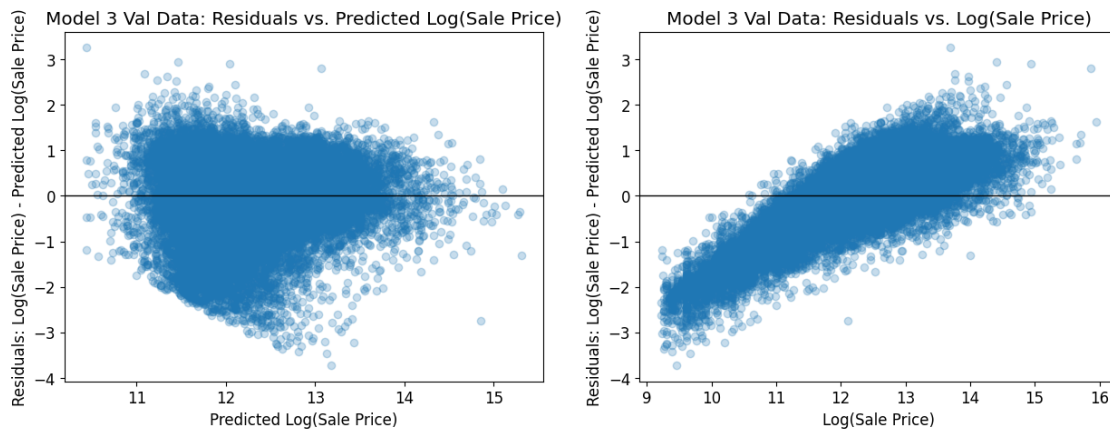
```

ax[0].set_title("Model 3 Val Data: Residuals vs. Predicted Log(Sale Price)")

ax[1].scatter(x_plt2, y_plt2, alpha=.25)
ax[1].axhline(0, c='black', linewidth=1)
ax[1].set_xlabel(r'Log(Sale Price)')
ax[1].set_ylabel(r'Residuals: Log(Sale Price) - Predicted Log(Sale Price)');
ax[1].set_title("Model 3 Val Data: Residuals vs. Log(Sale Price)")

```

Out[49]: Text(0.5, 1.0, 'Model 3 Val Data: Residuals vs. Log(Sale Price)')



0.1.4 Question 5c

- i). Comment on your RMSE and residual plots from Model 3 compared to the first 2 models.
 - ii). Are the residuals of your model still showing a trend that overestimates lower priced houses and underestimates higher priced houses? If so, how could you try to address this in the next round of modeling?
 - iii). If you had more time to improve your model, what would your next steps be?
-
- i) My RMSE and residual plots had improved to create a better model compared to the first two models. But both of these metrics had not changed by the amount that I had wanted them too. The RMSEs had all decrease, but the residual plot didn't change enough for a noticable difference.
 - ii) Yes it still has this trend. In the next round of modeling I could try and include a metric that is more closely tied to the sale price. But if this trend actually exists in the data we likely won't be able to fix this by adding more metrics.
 - iii) I would try and add more metrics to the model, or take some of the ones already in the model out of it. This could be because some of the metrics in the model currently might be a bad metric to use.

0.2 Question 6a

When evaluating your model, we used RMSE. In the context of estimating the value of houses, what does the residual mean for an individual homeowner? How does it affect them in terms of property taxes? Discuss the cases where residual is positive and negative separately.

The residual in the case of homeowners tells the difference between how much we expect them to pay vs how much they are actually paying in property tax. If we have a residual that is positive, this means that they are most likely paying less in property tax than they should be. This makes sense because their home price is lower than their expected home price from our model. And it works the same for a residual that is negative, as their home is being overvalued compared to our model. This means they are likely paying more in property taxes than they should be.

0.3 Question 6b

Reflecting back on your exploration in Questions 5 and 6a, in your own words, what makes a model's predictions of property values for tax assessment purposes "fair"?

This question is open-ended and part of your answer may depend upon your specific model; we are looking for thoughtfulness and engagement with the material, not correctness.

Hint: Some guiding questions to reflect on as you answer the question above: What is the relationship between RMSE, accuracy, and fairness as you have defined it? Is a model with a low RMSE necessarily accurate? Is a model with a low RMSE necessarily "fair"? Is there any difference between your answers to the previous two questions? And if so, why?

I would say that the current actual evaluations of home prices are not fair. And in order to make this model more fair, we should try and get a model that does have a low RMSE. This is because our predictions would better suit a more accurate property tax evaluation. A more accurate model would make for a more fair model since if it better predicts a certain home's price, it would generate a property tax that is actually closer to what they should be paying.

0.4 Extra Credit Step 1: Creating Your Model

Complete the modeling steps (you can skip the cross validation step to save memory) in the cells below.

DO NOT ADD ANY EXTRA CELLS BELOW (for this part of the problem)

```
In [93]: # Modeling Step 1: Process the Data
```

```
# Hint: You can either use your implementation of the One Hot Encoding Function from
#Project Part 1, or use the staff's implementation
from feature_func import *

...
# Optional: Define any helper functions you need (for example, for one-hot
#encoding, etc) above this line

def process_data_ec(data, is_test_set=False):
    # Please include all of your feature engineering processes for both
    #the training/validation as well as the test data inside this function.

    # Can include feature engineering processes for both the training/validation
    # and the test data above this line
    data_copy = data[(data["Pure Market Filter"] == 1)]

    # Whenever you access 'Log Sale Price' or 'Sale Price', make sure to use the
    # condition is_test_set like this:
    if not is_test_set:

        # Processing for the training/validation set (i.e. not the test set)
        # CAN involve references to sale price!
        # CAN involve filtering certain rows or removing outliers
        data_copy['Log Sale Price'] = np.log(data['Sale Price'])
        oh_enc = OneHotEncoder()
        oh_enc.fit(data_copy[["Roof Material"]]);
        arr = pd.DataFrame(oh_enc.transform(data_copy[["Roof Material"]]).todense(), columns =
        data_copy = data_copy.join(arr)

        data_copy["Log Buidling Square Feet"] = np.log(data_copy["Building Square Feet"])

        oh_enc.fit(data_copy[["Wall Material"]]);
        arr2 = pd.DataFrame(oh_enc.transform(data_copy[["Wall Material"]]).todense(), columns =
        data_copy = data_copy.join(arr2)
```



```

# Take a look at the result
display(X_train_ec.head())
display(Y_train_ec.head())

display(X_valid_ec.head())
display(Y_valid_ec.head())

```

	Log Buidling Square Feet	Roof Material_1.0	Roof Material_2.0	\
130829	7.870166	1.0	0.0	
193890	7.002156	1.0	0.0	
30507	6.851185	1.0	0.0	
91308	7.228388	1.0	0.0	
131132	7.990915	1.0	0.0	

	Roof Material_3.0	Roof Material_4.0	Roof Material_5.0	\
130829	0.0	0.0	0.0	
193890	0.0	0.0	0.0	
30507	0.0	0.0	0.0	
91308	0.0	0.0	0.0	
131132	0.0	0.0	0.0	

	Roof Material_6.0	Wall Material_1.0	Wall Material_2.0	\
130829	0.0	1.0	0.0	
193890	0.0	1.0	0.0	
30507	0.0	0.0	1.0	
91308	0.0	1.0	0.0	
131132	0.0	0.0	0.0	

	Wall Material_3.0	...	Garage 1 Size_2.0	Garage 1 Size_3.0	\
130829	0.0	...	0.0	1.0	
193890	0.0	...	0.0	0.0	
30507	0.0	...	0.0	1.0	
91308	0.0	...	0.0	1.0	
131132	1.0	...	0.0	1.0	

	Garage 1 Size_4.0	Garage 1 Size_5.0	Garage 1 Size_6.0	\
130829	0.0	0.0	0.0	
193890	0.0	0.0	0.0	
30507	0.0	0.0	0.0	
91308	0.0	0.0	0.0	
131132	0.0	0.0	0.0	

	Garage 1 Size_7.0	Porch_1	Porch_2	Porch_3	Age Cubed
130829	0.0	0.0	0.0	1.0	19683
193890	0.0	0.0	0.0	1.0	857375
30507	0.0	0.0	0.0	1.0	216000
91308	0.0	0.0	0.0	1.0	15625
131132	0.0	0.0	0.0	1.0	27000

[5 rows x 23 columns]

```

130829    12.994530
193890    11.848683
30507     11.813030
91308     13.060488
131132    12.516861
Name: Log Sale Price, dtype: float64

```

```

      Log Buidling Square Feet  Roof Material_1.0  Roof Material_2.0  \
50636                        7.310550           1.0           0.0
82485                        7.325808           1.0           0.0
193966                       7.090077           1.0           0.0
160612                       7.281386           1.0           0.0
7028                         7.118016           1.0           0.0

```

```

      Roof Material_3.0  Roof Material_4.0  Roof Material_5.0  \
50636                0.0                0.0                0.0
82485                0.0                0.0                0.0
193966               0.0                0.0                0.0
160612               0.0                0.0                0.0
7028                 0.0                0.0                0.0

```

```

      Roof Material_6.0  Wall Material_1.0  Wall Material_2.0  \
50636                0.0                1.0                0.0
82485                0.0                1.0                0.0
193966               0.0                0.0                1.0
160612               0.0                1.0                0.0
7028                 0.0                0.0                1.0

```

```

      Wall Material_3.0  ...  Garage 1 Size_2.0  Garage 1 Size_3.0  \
50636                0.0  ...                0.0                0.0
82485                0.0  ...                0.0                0.0
193966               0.0  ...                0.0                0.0
160612               0.0  ...                0.0                1.0
7028                 0.0  ...                0.0                1.0

```

```

      Garage 1 Size_4.0  Garage 1 Size_5.0  Garage 1 Size_6.0  \
50636                0.0                0.0                0.0
82485                0.0                0.0                0.0
193966               0.0                0.0                0.0
160612               0.0                0.0                0.0
7028                 0.0                0.0                0.0

```

```

      Garage 1 Size_7.0  Porch_1  Porch_2  Porch_3  Age Cubed
50636                0.0      0.0      0.0      1.0    1601613
82485                0.0      0.0      0.0      1.0    474552
193966               0.0      0.0      0.0      1.0    216000
160612               0.0      0.0      0.0      1.0    262144
7028                 0.0      0.0      0.0      1.0    804357

```

```
[5 rows x 23 columns]
```

```
50636    11.682668
```

```
82485      12.820655
193966      9.825526
160612     12.468437
7028       12.254863
Name: Log Sale Price, dtype: float64
```

```
In [94]: # Modeling STEP 2: Create a Multiple Linear Regression Model
```

```
# If you are are incorporating one-hot-encoded data, set the fit_intercept to False
```

```
linear_model_ec = lm.LinearRegression(fit_intercept=False)
```

```
linear_model_ec.fit(X_train_ec, Y_train_ec)
```

```
# your code above this line to create regression model for Model ec
```

```
Y_predict_train_ec = linear_model_ec.predict(X_train_ec)
```

```
Y_predict_valid_ec = linear_model_ec.predict(X_valid_ec)
```

```
In [95]: # MODELING STEP 3: Evaluate the RMSE for your model
```

```
# Training and test errors for the model
```

```
 #(in its original values before the log transform)
```

```
Y_predict_train_ec_transformed = np.exp(Y_predict_train_ec)
```

```
Y_train_ec_transformed = np.exp(Y_train_ec)
```

```
Y_predict_valid_ec_transformed = np.exp(Y_predict_valid_ec)
```

```
Y_valid_ec_transformed = np.exp(Y_valid_ec)
```

```
# Training and test errors for the model (in its original values before the log transform)
```

```
training_error_ec = rmse(Y_predict_train_ec_transformed, Y_train_ec_transformed.array)
```

```
validation_error_ec = rmse(Y_predict_valid_ec_transformed, Y_valid_ec_transformed.array)
```

```
(print("Extra Credit \nTraining RMSE: {}\nValidation RMSE: {}\n"  
      .format(training_error_ec, validation_error_ec))  
)
```

```
Extra Credit
```

```
Training RMSE: 239193.222900813
```

```
Validation RMSE: 244375.88824558211
```

In [96]: # Optional: Run this cell to visualize

```
#import plotly.graph_objects as go

fig = go.Figure([
go.Bar(x = ["Extra Credit Model"], y = [training_error_ec], name="Training RMSE"),
go.Bar(x = ["Extra Credit Model"], y = [validation_error_ec], name="Validation RMSE"),
])

fig
fig.update_yaxes(range=[140000,260000], title="RMSE")
# Feel free to update the range as needed
```



In [97]: # MODELING STEP 5: Plot 2 side-by-side residual plots for validation data

```
fig, ax = plt.subplots(1,2, figsize=(15, 5))

x_plt1 = Y_predict_valid_ec
y_plt1 = Y_valid_ec.array - Y_predict_valid_ec

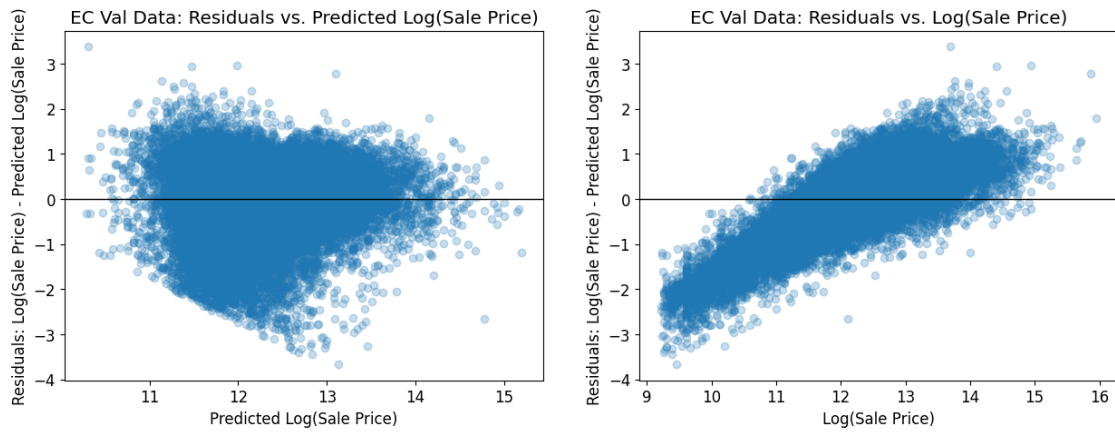
x_plt2 = Y_valid_ec.array
y_plt2 = Y_valid_ec.array - Y_predict_valid_ec

ax[0].scatter(x_plt1, y_plt1, alpha=.25)
ax[0].axhline(0, c='black', linewidth=1)
ax[0].set_xlabel(r'Predicted Log(Sale Price)')
ax[0].set_ylabel(r'Residuals: Log(Sale Price) - Predicted Log(Sale Price)');
ax[0].set_title("EC Val Data: Residuals vs. Predicted Log(Sale Price)")

ax[1].scatter(x_plt2, y_plt2, alpha=.25)
ax[1].axhline(0, c='black', linewidth=1)
ax[1].set_xlabel(r'Log(Sale Price)')
```

```
ax[1].set_ylabel(r'Residuals: Log(Sale Price) - Predicted Log(Sale Price)');  
ax[1].set_title("EC Val Data: Residuals vs. Log(Sale Price)")
```

Out[97]: Text(0.5, 1.0, 'EC Val Data: Residuals vs. Log(Sale Price)')



0.5 Extra Credit Step 2: Explanation (Required for points on model above):

Explain what you did to create your model. What versions did you try? What worked and what didn't?

Comment on the RMSE and residual plots from your model. Are the residuals of your model still showing a trend that overestimates lower priced houses and underestimates higher priced houses?

Type your answer here, replacing this text.

